

Sri Lanka Institute of Information Technology



IT3091 – Machine Learning

Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 03

Student ID	IT24103352
Student Name	M.A.W.AHAMED
Campus/ Center name	Kandy Uni
Specialization	AI

Part A – Data Cleaning

In [16]:

```
# Import libraries and load dataset
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import LabelEncoder, OneHotEncoder, OrdinalEncoder
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

In [17]:

```
df = sns.load_dataset("titanic")

print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	\
0	0	3	male	22.0	1	0	7.2500	S	Third	
1	1	1	female	38.0	1	0	71.2833	C	First	
2	1	3	female	26.0	0	0	7.9250	S	Third	
3	1	1	female	35.0	1	0	53.1000	S	First	
4	0	3	male	35.0	0	0	8.0500	S	Third	

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

In [18]:

```
# Identify feature types
target = "survived"

numerical_features = df.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = df.select_dtypes(include=["object", "category", "bool"]).columns.tolist()

numerical_features.remove(target)

print("Numerical features:", numerical_features)
print("Categorical features:", categorical_features)
print("Target variable:", target)
```

```
Numerical features: ['pclass', 'age', 'sibsp', 'parch', 'fare']
Categorical features: ['sex', 'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town', 'alive', 'alone']
Target variable: survived
```

In [19]:

```
# Check missing values and duplicates
print("Missing values:")
print(df.isnull().sum())

print("\nDuplicate records:")
print(df.duplicated().sum())

df = df.drop_duplicates()
```

```
print("\nUpdated shape after removing duplicates:")
print(df.shape)
```

```
Missing values:
survived      0
pclass        0
sex           0
age          177
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone         0
dtype: int64
```

```
Duplicate records:
107
```

```
Updated shape after removing duplicates:
(784, 15)
```

In [20]:

```
# Missing values before imputation
missing_before = df.isnull().sum()
print("Missing values before imputation:")
print(missing_before[missing_before > 0])
```

```
Missing values before imputation:
age          106
embarked      2
deck         582
embark_town   2
dtype: int64
```

In [21]:

```
# Imputation
df_imputed = df.copy()

df_imputed["age"] = df_imputed["age"].fillna(df_imputed["age"].median())
df_imputed["embarked"] = df_imputed["embarked"].fillna(df_imputed["embarked"].mode()[0])
df_imputed["embark_town"] = df_imputed["embark_town"].fillna(df_imputed["embark_town"].mode()[0])
df_imputed["deck"] = df_imputed["deck"].fillna(df_imputed["deck"].mode()[0])
```

In [22]:

```
#Since age is a numerical feature, median imputation is used to fill its missing values.
Median is suitable because it is less affected by outliers than the mean.

#Since embarked, embark_town, and deck are categorical features, mode imputation is used
to fill their missing values. Mode is suitable because it replaces missing values with the
most frequently occurring category.
```

In [23]:

```
print("Missing values after imputation:")
print(df_imputed.isnull().sum()[df_imputed.isnull().sum() > 0])
```

```
Missing values after imputation:
Series([], dtype: int64)
```

Part B – Categorical Data Encoding

In [24]:

```
# Identify categorical feature types target = "survived" categorical_features =
df_imputed.select_dtypes(include=["object", "category", "bool"]). columns.tolist()

binary_categorical = [col for col in categorical_features if df_imputed[col].nunique() =
= 2]

nominal_categorical = ["embarked", "embark_town", "who", "adult_male", "alone"]
ordinal_categorical = ["class", "deck"]

print("Binary categorical features:", binary_categorical)
print("Nominal categorical features:", nominal_categorical) print("Ordinal
categorical features:", ordinal_categorical)

Binary categorical features: ['sex', 'adult_male', 'alive', 'alone']
Nominal categorical features: ['embarked', 'embark_town', 'who', 'adult_male', 'alone']
Ordinal categorical features: ['class', 'deck']
```

In [25]:

```
# Encoding encoded_df = df_imputed.copy()

# Label encode binary categorical features
label_encoder = LabelEncoder()

for col in binary_categorical:    encoded_df[col] =
label_encoder.fit_transform(encoded_df[col])

# One-hot encode nominal categorical features encoded_df = pd.get_dummies(encoded_df,
columns=nominal_categorical, drop_first=True)

# Ordinal encode class class_order = ["First",
"Second",      "Third"]    class_encoder      =
OrdinalEncoder(categories=class_order) encoded_df[["class"]] =
class_encoder.fit_transform(encoded_df[["class"]])

# Ordinal encode deck deck_order = ["A", "B", "C",
"D",      "E",      "F",      "G"]    deck_encoder      =
OrdinalEncoder(categories=deck_order) encoded_df[["deck"]]      =
deck_encoder.fit_transform(encoded_df[["deck"]])

print(encoded_df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	class	deck	alive	\ 0
0	3	1	22.0	1	0	7.2500	2.0	2.0	0		
1	1	1	0	38.0	1	0	71.2833	0.0	2.0	1	
2	1	3	0	26.0	0	0	7.9250	2.0	2.0	1	
3	1	1	0	35.0	1	0	53.1000	0.0	2.0	1	
4	0	3	1	35.0	0	0	8.0500	2.0	2.0	0	

	embarked_Q	embarked_S	embark_town_Queenstown	embark_town_Southampton	\
0	False	True	False		True
1	False	False	False		False
2	False	True	False		True
3	False	True	False		True
4	False	True	False		True

	who_man	who_woman	adult_male_1	alone_1	0
True	False		True	False	
1	False	True	False	False	
2	False	True	False	True	
3	False	True	False	False	
4	True	False	True	True	

In [28]:

```
#Binary categorical features such as sex, alive, adult_male, and alone are encoded using
```

label encoding because they contain only two categories.

#Nominal categorical features such as embarked, embark_town, and who are encoded using one-hot encoding because they do not have a natural order.

#Ordinal categorical features such as class and deck are encoded using ordinal encoding because their categories have a meaningful order.

#The transformed dataset is displayed after applying these encoding techniques.

Part C - Feature Scaling

In [26]:

```
# Numerical data
num_data = df_imputed[numerical_features]

standard_scaled = pd.DataFrame(
    StandardScaler().fit_transform(num_data),
    columns=numerical_features
)

minmax_scaled = pd.DataFrame(
    MinMaxScaler().fit_transform(num_data),
    columns=numerical_features
)

robust_scaled = pd.DataFrame(
    RobustScaler().fit_transform(num_data),
    columns=numerical_features
)

print("StandardScaler:")
print(standard_scaled.head())

print("\nMinMaxScaler:")
print(minmax_scaled.head())

print("\nRobustScaler:")
print(robust_scaled.head())
```

StandardScaler:

	pclass	age	sibsp	parch	fare
0	0.885158	-0.557360	0.484009	-0.497157	-0.526825
1	-1.455362	0.608298	0.484009	-0.497157	0.701587
2	0.885158	-0.265945	-0.530599	-0.497157	-0.513876
3	-1.455362	0.389737	0.484009	-0.497157	0.352760
4	0.885158	0.389737	-0.530599	-0.497157	-0.511478

MinMaxScaler:

	pclass	age	sibsp	parch	fare
0	1.0	0.271174	0.125	0.0	0.014151
1	0.0	0.472229	0.125	0.0	0.139136
2	1.0	0.321438	0.000	0.0	0.015469
3	0.0	0.434531	0.125	0.0	0.103644
4	1.0	0.434531	0.000	0.0	0.015713

RobustScaler:

	pclass	age	sibsp	parch	fare
0	0.0	-0.446429	1.0	0.0	-0.331935
1	-1.0	0.696429	1.0	0.0	2.125276
2	0.0	-0.160714	0.0	0.0	-0.306032
3	-1.0	0.482143	1.0	0.0	1.427511
4	0.0	0.482143	0.0	0.0	-0.301235

In [27]:

```
print("StandardScaler mean:")
print(standard_scaled.mean())

print("\nMinMaxScaler minimum:")
```

```
print(minmax_scaled.min())

print("\nMinMaxScaler maximum:")
print(minmax_scaled.max())

print("\nRobustScaler median:")
print(robust_scaled.median())
```

```
StandardScaler mean:
pclass    -1.631348e-16
age        1.540718e-16
sibsp     -4.984675e-17
parch      2.039185e-17
fare      -6.910572e-17
dtype: float64
```

```
MinMaxScaler minimum:
pclass     0.0
age         0.0
sibsp       0.0
parch       0.0
fare        0.0
dtype: float64
```

```
MinMaxScaler maximum:
pclass     1.0
age         1.0
sibsp       1.0
parch       1.0
fare        1.0
dtype: float64
```

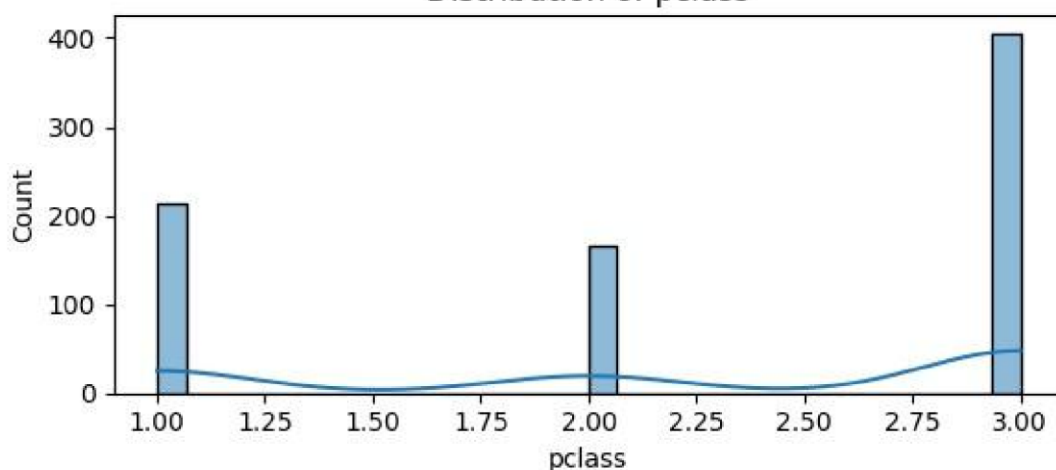
```
RobustScaler median:
pclass     0.0
age         0.0
sibsp       0.0
parch       0.0
fare        0.0
dtype: float64
```

Part D – Feature transformation

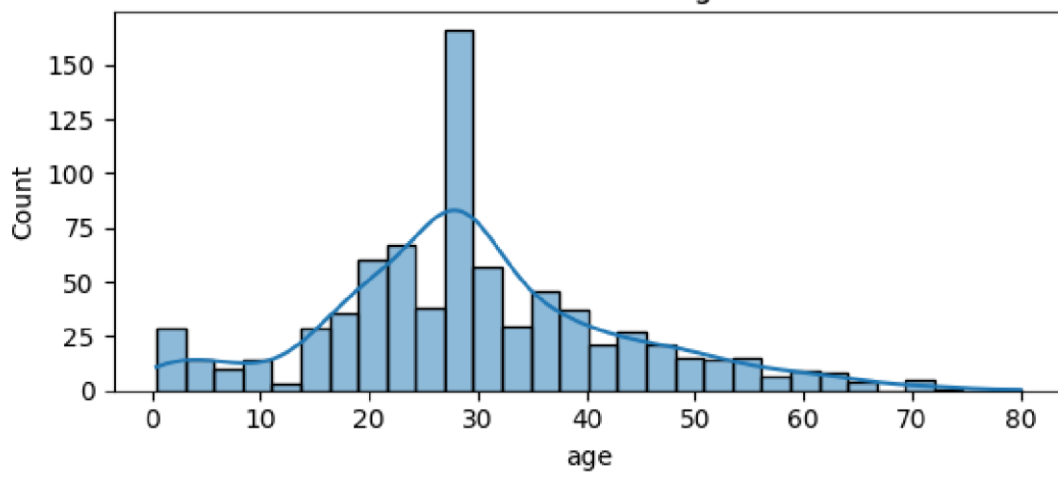
In [29]:

```
# Histograms to identify skewness
for col in numerical_features:
    plt.figure(figsize=(6, 3))
    sns.histplot(df_imputed[col], kde=True, bins=30)
    plt.title("Distribution of " + col)
    plt.tight_layout()
    plt.show()
```

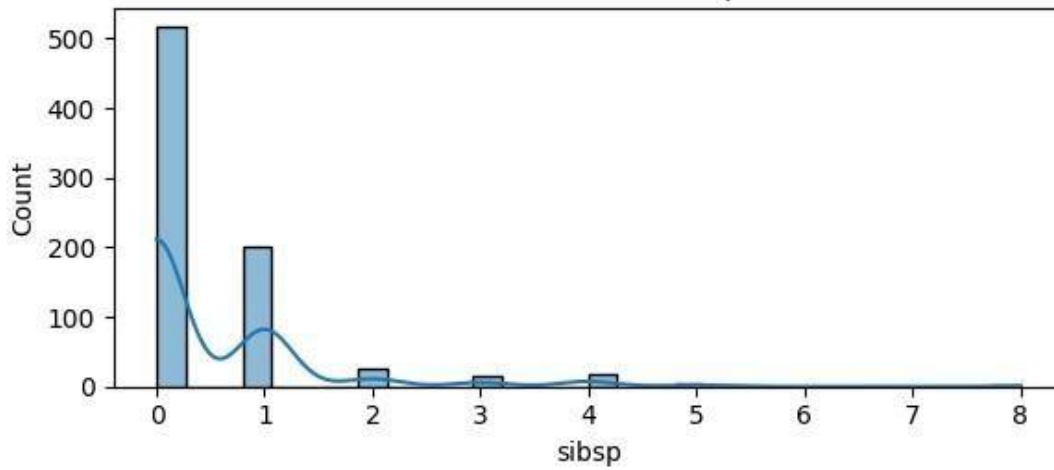
Distribution of pclass



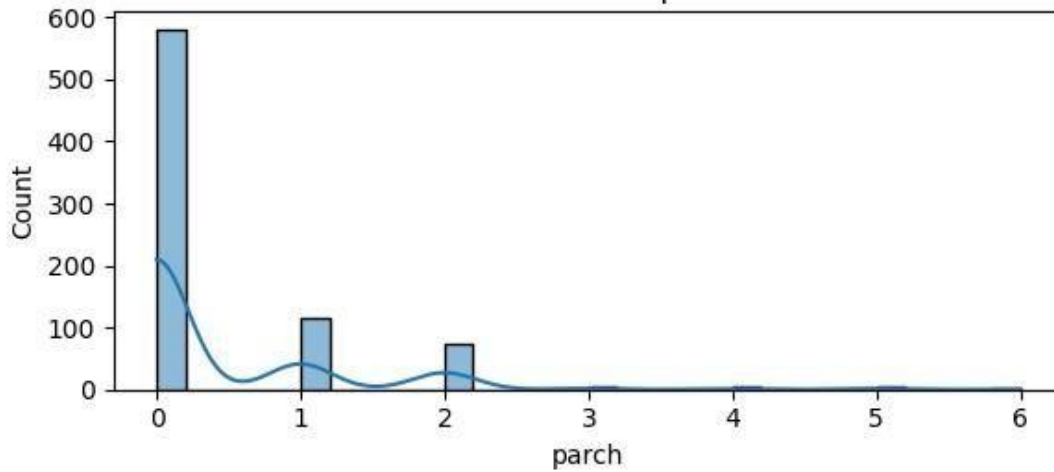
Distribution of age



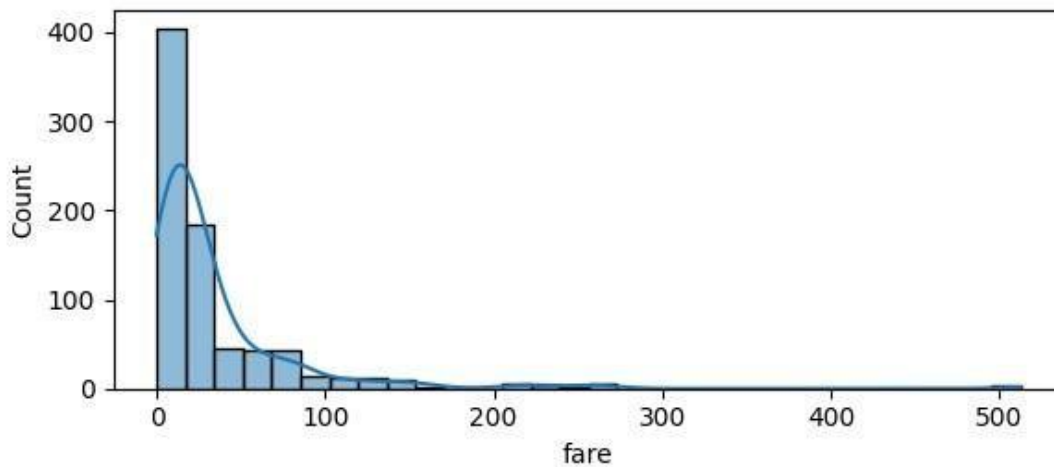
Distribution of sibsp



Distribution of parch



Distribution of fare



```
In [30]:
```

```
# Check skewness
print(df_imputed[numerical_features].skew())
```

```
pclass    -0.488966
age         0.441779
sibsp       3.055367
parch       2.633002
fare        4.570768
dtype: float64
```

```
In [31]:
```

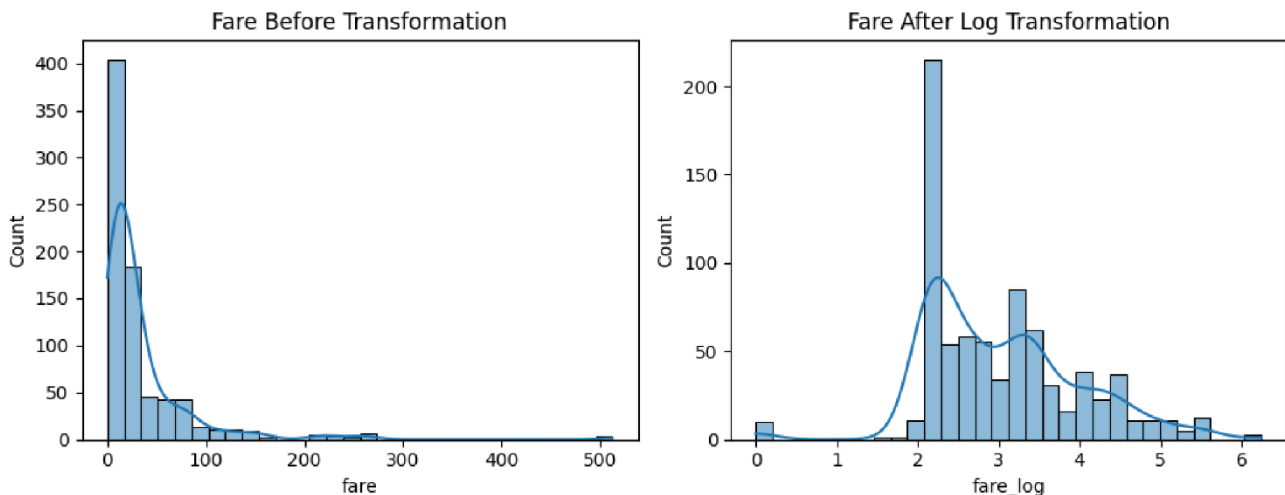
```
# Apply log transformation to skewed feature: fare
df_transformed = df_imputed.copy()
df_transformed["fare_log"] = np.log1p(df_transformed["fare"])

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
sns.histplot(df_imputed["fare"], kde=True, bins=30)
plt.title("Fare Before Transformation")

plt.subplot(1, 2, 2)
sns.histplot(df_transformed["fare_log"], kde=True, bins=30)
plt.title("Fare After Log Transformation")

plt.tight_layout()
plt.show()
```



```
In [32]:
```

```
X = df.drop(columns=["survived"])
y = df["survived"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

numeric_features = X_train.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = X_train.select_dtypes(include=["object", "category", "bool"]).columns.tolist()

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", RobustScaler())
])

categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

```
preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

print("Training data shape:", X_train_processed.shape)
print("Testing data shape:", X_test_processed.shape)
```

Training data shape: (627, 32)

Testing data shape: (157, 32)

Part E – Case Study

1 What is the main difference between the two preprocessing workflows?

The main difference is the order of preprocessing and train-test splitting. In the first workflow, missing value imputation and feature scaling are applied to the entire dataset before splitting. In the second workflow, the dataset is first split into training and testing sets, then preprocessing is fitted only on the training dataset and applied to both training and testing datasets.

1 Which workflow introduces data leakage?

The first workflow introduces data leakage because preprocessing is performed using the entire dataset, including the test data.

1 Why Should Preprocessing Be Fitted Only On Training Data?

Preprocessing methods should be fitted only on the training dataset because the test dataset must represent unseen data. If information from the test data is used to calculate values such as mean, median, mode, minimum, maximum, or standard deviation, the model indirectly learns information from the test set. This can produce unrealistically high performance.

1 Which workflow would you recommend for building a reliable machine learning model? Justify your answer.

The second workflow is recommended for building a reliable machine learning model. The dataset should be split first, then preprocessing should be fitted only on the training data. The same fitted preprocessing steps should then be used to transform both training and testing data. This prevents data leakage and gives a more realistic estimate of model performance on unseen data.